Servo motor Code

```
import time

import os


PWM_CHIP    = "/sys/class/pwm/pwmchip13"

PWM_CHANNEL = 0

PWM_PATH    = f"{PWM_CHIP}/pwm{PWM_CHANNEL}"

PERIOD_NS   = 20_000_000


def pwm_write(path, val):

    with open(path, "w") as f:

        f.write(str(val))


def angle_to_duty(angle):

    pulse_ms = 0.5 + (angle / 180.0) * 2.0

    return int(pulse_ms * 1_000_000)


# Setup

if not os.path.exists(PWM_PATH):

    pwm_write(f"{PWM_CHIP}/export", PWM_CHANNEL)

    time.sleep(0.2)


pwm_write(f"{PWM_PATH}/period",     PERIOD_NS)

pwm_write(f"{PWM_PATH}/polarity",   "normal")   # ← FIX added

pwm_write(f"{PWM_PATH}/duty_cycle", angle_to_duty(90))

pwm_write(f"{PWM_PATH}/enable",     1)
```

```
print("Hardware PWM ready ✓")


def move_servo(angle):
    print(f"→ {angle}°")
    pwm_write(f"{PWM_PATH}/duty_cycle", angle_to_duty(angle))
    time.sleep(0.6)
    pwm_write(f"{PWM_PATH}/duty_cycle", 0)  # silence = no vibration


try:
    while True:
        for angle in range(0, 181, 20):
            move_servo(angle)
            time.sleep(1)
        for angle in range(180, -1, -20):
            move_servo(angle)
            time.sleep(1)


except KeyboardInterrupt:
    print("\nStopped")
finally:
    pwm_write(f"{PWM_PATH}/enable",   0)
    pwm_write(f"{PWM_CHIP}/unexport", PWM_CHANNEL)
    print("PWM released ✓")
```

HLK LD 2451 codes

Arduino Code :

```
import serial

import serial.tools.list_ports

import tkinter as tk

import matplotlib.pyplot as plt

import matplotlib.gridspec as gridspec

from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

from matplotlib.animation import FuncAnimation

import collections


# ────────────────────────────────────────────────

# PORT SETUP

# ────────────────────────────────────────────────

PORT = "/dev/radar"   # fixed via udev rule

BAUD = 115200


try:

    ser = serial.Serial(PORT, BAUD, timeout=1)

    print(f"Connected: {PORT}")

except Exception:

    # Fallback: auto detect

    ser = None

    for p in serial.tools.list_ports.comports():

        if 'ACM' in p.device or 'USB' in p.device:

            try:

                ser = serial.Serial(p.device, BAUD, timeout=1)
```

```
            PORT = p.device
            print(f"Fallback port: {PORT}")
            break
        except Exception:
            continue
    if ser is None:
        print("ERROR: No Arduino port found!")
        exit(1)

# ────────────────────────────────────────────────
# DATA BUFFERS
# ────────────────────────────────────────────────
MAX_POINTS = 100
dist_data  = collections.deque([0] * MAX_POINTS, maxlen=MAX_POINTS)
speed_data = collections.deque([0] * MAX_POINTS, maxlen=MAX_POINTS)
snr_data   = collections.deque([0] * MAX_POINTS, maxlen=MAX_POINTS)

# ────────────────────────────────────────────────
# WINDOW SETUP
# ────────────────────────────────────────────────
root = tk.Tk()
root.title("HLK-LD2451 Radar Monitor")
root.configure(bg="#0a0a0a")

# Left half of screen
```

```
sw = root.winfo_screenwidth()
sh = root.winfo_screenheight()
root.geometry(f"{sw // 2}x{sh}+0+0")

# ────────────────────────────────────────────────
# INFO BOX
# ────────────────────────────────────────────────
info_frame = tk.Frame(root, bg="#0a0a0a", pady=6)
info_frame.pack(fill="x", padx=12)

tk.Label(info_frame,
        text="━━━
```

```
# Field definitions: (label, color)
fields = [
    ("Alarm",     "#ffffff"),
    ("Angle",     "#00cfff"),
    ("Distance",  "#ffff00"),
    ("Direction", "#ff4444"),
    ("Speed",     "#ff9900"),
    ("SNR",       "#cc88ff"),
]

labels = {}
for field, color in fields:
    row = tk.Frame(info_frame, bg="#0a0a0a")
    row.pack(anchor="w", pady=2)
    tk.Label(row,
             text=f"  {field:<12}: ",
             bg="#0a0a0a", fg="#888888",
             font=("Courier", 12)).pack(side="left")
    labels[field] = tk.Label(row,
             text="---",
             bg="#0a0a0a", fg=color,
             font=("Courier", 12, "bold"))
    labels[field].pack(side="left")

tk.Label(info_frame,
```

```
        text="━━━
```

```
    ax.set_ylabel(ylabel, color=color, fontsize=7)
    ax.tick_params(colors="#444444", labelsize=6)
    ax.set_ylim(*ylim)
    ax.set_xlim(0, MAX_POINTS)
    ax.grid(True, color="#1e1e1e", linewidth=0.5)
    for sp in ax.spines.values():
        sp.set_color("#2a2a2a")
    line, = ax.plot([], [], color=color, linewidth=1.3)
    return ax, line

ax1, line_dist  = make_axis(gs[0], "Distance",       "m",    "#ffff00", (0, 50))
ax2, line_speed = make_axis(gs[1], "Speed",          "km/h", "#ff9900", (0, 30))
ax3, line_snr   = make_axis(gs[2], "Signal Strength", "SNR",  "#cc88ff", (0, 255))

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill="both", expand=True, padx=6, pady=4)

# ────────────────────────────────────────────────────────
# UPDATE FUNCTION
# ────────────────────────────────────────────────────────
def update(_):
    try:
        updated = False
        while ser.in_waiting:
            raw   = ser.readline().decode(errors="ignore").strip()
```

```
parts = raw.split(",")

if len(parts) != 6:
    continue

alarm, angle, distance, direction, speed, snr = parts

try:
    alarm_int = int(alarm)
    dist_f    = float(distance)
    speed_f   = float(speed)
    snr_f     = float(snr)
except ValueError:
    continue

if alarm_int > 1:
    continue

# Update info labels
dir_color = "#ff4444" if direction == "Approaching" else "#00ff88"
labels["Alarm"].config(text=alarm)
labels["Angle"].config(text=f"{angle} deg")
labels["Distance"].config(text=f"{distance} m")
labels["Direction"].config(text=direction, fg=dir_color)
labels["Speed"].config(text=f"{speed} km/h")
```

```
        labels["SNR"].config(text=snr)

        # Status indicator
        arrow = "►►" if direction == "Approaching" else "◄◄"
        status_var.set(
            f"  {arrow}  Target {direction}  |  "
            f"{distance} m  |  {speed} km/h"
        )

        # Push to buffers
        dist_data.append(dist_f)
        speed_data.append(speed_f)
        snr_data.append(snr_f)
        updated = True

    if updated:
        xdata = list(range(MAX_POINTS))
        line_dist.set_data(xdata,  list(dist_data))
        line_speed.set_data(xdata, list(speed_data))
        line_snr.set_data(xdata,   list(snr_data))
        canvas.draw_idle()

except Exception:
    pass
```

```
    return line_dist, line_speed, line_snr


# ————————————————————————————————————————————
# RUN
# ————————————————————————————————————————————
ani = FuncAnimation(fig, update, interval=100, blit=False)
root.mainloop()
```

Python Code :

```
import serial
import serial.tools.list_ports
import tkinter as tk
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.animation import FuncAnimation
import collections
import time


# —— PORT ————————————————————————————————————————
PORT = "/dev/radar"
BAUD = 115200


try:
    ser = serial.Serial(PORT, BAUD, timeout=1)
    print(f"Connected: {PORT}")
except:
```

```
    ports = serial.tools.list_ports.comports()
    ser = None
    for p in ports:
        if 'ACM' in p.device or 'USB' in p.device:
            ser = serial.Serial(p.device, BAUD, timeout=1)
            print(f"Fallback: {p.device}")
            break
    if ser is None:
        print("ERROR: No port found!")
        exit(1)

# ── DATA BUFFERS ──────────────────────────────────────
MAX_POINTS = 100
dist_data  = collections.deque([0]*MAX_POINTS, maxlen=MAX_POINTS)
speed_data = collections.deque([0]*MAX_POINTS, maxlen=MAX_POINTS)
snr_data   = collections.deque([0]*MAX_POINTS, maxlen=MAX_POINTS)
time_data  = collections.deque(range(MAX_POINTS), maxlen=MAX_POINTS)
t = [0]

# ── WINDOW ────────────────────────────────────────────
root = tk.Tk()
root.title("HLK-LD2451 Radar")
root.configure(bg="#0a0a0a")

# Half screen left side
```

```
sw = root.winfo_screenwidth()

sh = root.winfo_screenheight()

w  = sw // 2

h  = sh

root.geometry(f"{w}x{h}+0+0")


# ── INFO BOX (top quarter) ──────────────────────────

info_frame = tk.Frame(root, bg="#0a0a0a", pady=5)

info_frame.pack(fill="x", padx=10)


tk.Label(info_frame,

        text="────────────────────────────────────────────────────────────────────────",

        bg="#0a0a0a", fg="#00ff88",

        font=("Courier", 10)).pack(anchor="w")


tk.Label(info_frame,

        text="  HLK-LD2451 RADAR MONITOR",

        bg="#0a0a0a", fg="#00ff88",

        font=("Courier", 12, "bold")).pack(anchor="w")


tk.Label(info_frame,

        text="────────────────────────────────────────────────────────────────────────",

        bg="#0a0a0a", fg="#00ff88",

        font=("Courier", 10)).pack(anchor="w")
```

```
labels = {}
field_config = [
    ("Alarm",     "#ffffff"),
    ("Angle",     "#00cfff"),
    ("Distance",  "#ffff00"),
    ("Direction", "#ff4444"),
    ("Speed",     "#ff9900"),
    ("SNR",       "#cc88ff"),
]

for field, color in field_config:
    row = tk.Frame(info_frame, bg="#0a0a0a")
    row.pack(anchor="w", pady=1)
    tk.Label(row,
             text=f"  {field:<12}: ",
             bg="#0a0a0a", fg="#888888",
             font=("Courier", 11)).pack(side="left")
    labels[field] = tk.Label(row,
             text="---",
             bg="#0a0a0a", fg=color,
             font=("Courier", 11, "bold"))
    labels[field].pack(side="left")

tk.Label(info_frame,
         text="━━━
```

```
        bg="#0a0a0a", fg="#00ff88",
        font=("Courier", 10)).pack(anchor="w")

# status bar
status_var = tk.StringVar(value="Waiting for target...")
tk.Label(info_frame,
        textvariable=status_var,
        bg="#0a0a0a", fg="#888888",
        font=("Courier", 9)).pack(anchor="w", pady=2)

# ── GRAPH (bottom quarter) ──────────────────────────
fig = plt.Figure(figsize=(w/100, h*0.55/100), dpi=100)
fig.patch.set_facecolor("#0a0a0a")

gs = gridspec.GridSpec(3, 1, figure=fig,
                    hspace=0.6,
                    top=0.92, bottom=0.1)

# Distance plot
ax1 = fig.add_subplot(gs[0])
ax1.set_facecolor("#111111")
ax1.set_ylabel("Dist (m)", color="#ffff00", fontsize=7)
ax1.set_title("Distance", color="#ffff00", fontsize=8, pad=3)
ax1.tick_params(colors="#555555", labelsize=6)
ax1.set_ylim(0, 50)
```

```
ax1.grid(True, color="#222222", linewidth=0.5)
for sp in ax1.spines.values():
    sp.set_color("#333333")
line_dist, = ax1.plot([], [], color="#ffff00",
                      linewidth=1.2, label="Distance")
ax1.fill_between(range(MAX_POINTS), 0, 0,
                 alpha=0.15, color="#ffff00")

# Speed plot
ax2 = fig.add_subplot(gs[1])
ax2.set_facecolor("#111111")
ax2.set_ylabel("Speed (km/h)", color="#ff9900", fontsize=7)
ax2.set_title("Speed", color="#ff9900", fontsize=8, pad=3)
ax2.tick_params(colors="#555555", labelsize=6)
ax2.set_ylim(0, 30)
ax2.grid(True, color="#222222", linewidth=0.5)
for sp in ax2.spines.values():
    sp.set_color("#333333")
line_speed, = ax2.plot([], [], color="#ff9900",
                       linewidth=1.2)

# SNR plot
ax3 = fig.add_subplot(gs[2])
ax3.set_facecolor("#111111")
ax3.set_ylabel("SNR", color="#cc88ff", fontsize=7)
```

```
ax3.set_title("Signal Strength", color="#cc88ff", fontsize=8, pad=3)
ax3.tick_params(colors="#555555", labelsize=6)
ax3.set_ylim(0, 255)
ax3.grid(True, color="#222222", linewidth=0.5)
for sp in ax3.spines.values():
    sp.set_color("#333333")
line_snr, = ax3.plot([], [], color="#cc88ff",
                     linewidth=1.2)

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(fill="both", expand=True, padx=5)

# ── UPDATE FUNCTION ──────────────────────────────────────
def update(_):
    try:
        while ser.in_waiting:
            raw = ser.readline().decode(errors="ignore").strip()
            parts = raw.split(",")

            if len(parts) != 6:
                continue

            alarm, angle, distance, direction, speed, snr = parts

            if int(alarm) > 1:
```

```
    continue

# Update labels
dir_color = "#ff4444" if direction == "Approaching" else "#00ff88"
labels["Alarm"].config(text=alarm)
labels["Angle"].config(text=f"{angle} deg")
labels["Distance"].config(text=f"{distance} m")
labels["Direction"].config(text=direction, fg=dir_color)
labels["Speed"].config(text=f"{speed} km/h")
labels["SNR"].config(text=snr)

# Status
status_var.set(
    f"Target detected  |  "
    f"{'⍰' if direction=='Approaching' else '⍰'} "
    f"{direction}"
)

# Update buffers
t[0] += 1
dist_data.append(float(distance))
speed_data.append(float(speed))
snr_data.append(float(snr))
time_data.append(t[0])
```

```
        xdata = list(range(MAX_POINTS))

        # Update plots
        line_dist.set_data(xdata, list(dist_data))
        line_speed.set_data(xdata, list(speed_data))
        line_snr.set_data(xdata, list(snr_data))

        for ax in [ax1, ax2, ax3]:
            ax.set_xlim(0, MAX_POINTS)

        canvas.draw_idle()

    except Exception as e:
        pass

    return line_dist, line_speed, line_snr

ani = FuncAnimation(fig, update, interval=100, blit=False)
root.mainloop()
```

YOLO code

```
import cv2
import numpy as np
import time
import threading
```

```
# —— CONFIG ————————————————————————————————————

BLOB_SIZE   = 160

CONF_THRESH = 0.45

NMS_THRESH  = 0.4

CAM_W, CAM_H = 320, 240      # ← Camera resolution bhi kam ki

ALLOWED = {"person", "bicycle", "car", "motorbike", "bus", "truck", “drone”,”aeroplane”}

# ————————————————————————————————————————————


net = cv2.dnn.readNet("yolov3-tiny.weights", "yolov3-tiny.cfg")

net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)

net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)


with open("coco.names") as f:

    classes = [l.strip() for l in f]


layer_names = net.getLayerNames()

out_layers  = [layer_names[i - 1] for i in net.getUnconnectedOutLayers().flatten()]


# —— THREADED CAMERA ————————————————————————————

class CameraStream:

    def __init__(self, src=0):

        self.cap = cv2.VideoCapture(src)

        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH,  CAM_W)

        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, CAM_H)
```

```
        self.cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
        self.ret, self.frame = self.cap.read()
        self.lock    = threading.Lock()
        self.running = True
        threading.Thread(target=self._update, daemon=True).start()

    def _update(self):
        while self.running:
            ret, frame = self.cap.read()
            with self.lock:
                self.ret, self.frame = ret, frame

    def read(self):
        with self.lock:
            return self.ret, self.frame.copy() if self.ret else (False, None)

    def stop(self):
        self.running = False
        self.cap.release()

# ── DETECT FUNCTION ──────────────────────────────────────
def detect(frame):
    h, w = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(frame, 1/255.0,
                                 (BLOB_SIZE, BLOB_SIZE),
```

```
                        swapRB=True, crop=False)
net.setInput(blob)
outs = net.forward(out_layers)


boxes, confs, cids = [], [], []
for out in outs:
    for det in out:
        scores = det[5:]
        cid  = int(np.argmax(scores))
        conf = float(scores[cid])
        if conf > CONF_THRESH and classes[cid] in ALLOWED:
            cx, cy, bw, bh = det[:4]
            x = int((cx - bw/2) * w)
            y = int((cy - bh/2) * h)
            boxes.append([x, y, int(bw*w), int(bh*h)])
            confs.append(conf)
            cids.append(cid)


idxs = cv2.dnn.NMSBoxes(boxes, confs, CONF_THRESH, NMS_THRESH)
results = []
if len(idxs) > 0:
    for i in idxs.flatten():
        results.append((boxes[i], classes[cids[i]], confs[i]))
return results
```

```
# —— MAIN LOOP ————————————————————————————————————

cam = CameraStream(0)

time.sleep(1)


detections  = []

fps_display = 0.0


# ← Ye sahi FPS measure karta hai (inference time)

infer_times = []


print(f"Running | Blob={BLOB_SIZE}x{BLOB_SIZE} | Press Q to quit")


while True:

    ret, frame = cam.read()

    if not ret:

        break


    # — YOLO inference + sahi time measure —

    t1 = time.time()

    detections = detect(frame)

    t2 = time.time()


    infer_ms = (t2 - t1) * 1000          # milliseconds

    infer_times.append(t2 - t1)

    if len(infer_times) > 20:
```

```
        infer_times.pop(0)

    # Real FPS = 1 / average inference time
    real_fps = 1.0 / (sum(infer_times) / len(infer_times))

    # — Draw detections —
    for (x, y, bw, bh), label, conf in detections:
        cv2.rectangle(frame, (x,y), (x+bw, y+bh), (0,255,0), 2)
        cv2.putText(frame, f"{label} {conf:.2f}",
                    (x, y-8), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)

    # — Show REAL FPS + inference time —
    cv2.putText(frame, f"Real FPS : {real_fps:.1f}",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255), 2)
    cv2.putText(frame, f"Infer ms : {infer_ms:.0f}ms",
                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255,128,0), 2)

    cv2.imshow("YOLO Fast - Radxa Zero 3W", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cam.stop()
cv2.destroyAllWindows()
print(f"Final Real FPS: {1.0/(sum(infer_times)/len(infer_times)):.1f}")
```